Incident Analysis & Remediation Report

Kaan Kadir Aluçlu, October 16, 2024

Section 1: Incident Analysis

1.1 Timeline Reconstruction (UTC Normalized)

The attack occurred in two primary phases, originating from the IP 203.0.113.45 and utilizing credentials for user_id=1523.

Phase 1: API Compromise & Data Exfiltration

- **06:45:10 UTC:** Attacker authenticates to /api/v1/login using a stolen JWT for user_1523.
    - 2024-10-15 06:45:10,1523,/api/v1/login,POST,,200,267,203.0.113.45,Acme-Mobile-Android/3.2.0,
- **06:46:30 - 06:47:57 UTC:** Attacker exploits a Broken Access Control (IDOR) vulnerability, iterating through user IDs (1524 to 1538) to exfiltrate 16 users' portfolio data.
    - 2024-10-15 06:47:15,1523,/api/v1/portfolio/1524,GET,1524,200,143,203.0.113.45,Acme-Mobile-Android/3.2.0,jwt_token_1523_stolen
    - ... (requests for 1525-1537) ...
    - 2024-10-15 06:47:57,1523,/api/v1/portfolio/1538,GET,1538,200,147,203.0.113.45,Acme-Mobile-Android/3.2.0,jwt_token_1523_stolen

Phase 2: Web Application Compromise & Data Exfiltration

- **09:00:23 UTC:** Attacker initiates a parallel phishing campaign (likely the vector for the initial token theft) from the same IP.
    - 2024-10-15 09:00:23,security@acme-finance.com,user1@acme.com,URGENT: Verify Your Account...,yes,203.0.113.45
- **09:20:30 - 09:22:00 UTC:** Attacker, using the *same user identity* (user_1523), probes the /dashboard/search endpoint with standard SQLi payloads. These are successfully **BLOCKED** by the WAF.
    - 2024-10-15 09:21:15,981318,CRITICAL,BLOCK,203.0.113.45,/dashboard/search,SQL Injection - DROP TABLE,yes
- **09:23:45 UTC (CRITICAL):** Attacker executes a WAF-bypass SQLi payload (/*!50000OR*/ 1=1--). The WAF (Rule 981001) detects this as "Medium" risk but is misconfigured to **DETECT-ONLY (Block: no)**. The server responds with 200 OK, confirming the bypass.
    - 2024-10-15 09:23:45,1523,/dashboard/search,ticker=AAPL' /*!50000OR*/ 1=1--,200,156789,203.0.113.45,...
    - 2024-10-15 09:23:45,981001,MEDIUM,DETECT,203.0.113.45,/dashboard/search,Suspicious SQL Pattern,no

- **09:24:10 UTC:** Attacker immediately pivots to the /dashboard/export function and exfiltrates a large (892KB) CSV file.
    - 2024-10-15 09:24:10,1523,/dashboard/export,format=csv,200,892341,203.0.113.45,...
- **09:30:00 UTC:** Attacker accesses /dashboard/home to verify session persistence before logging off.

1.2 Attack Vector Identification & Classification

The attack utilized a chain of vulnerabilities, classified below.

OWASP Top 10 (2021):

- **A01:2021 - Broken Access Control:** This is the root cause of the API breach. The /api/v1/portfolio/ endpoint checked for authentication (a valid token) but failed to perform authorization (checking if user_1523 was allowed to see user_1524's data).
- **A03:2021 - Injection:** This is the root cause of the web breach. The ticker parameter in /dashboard/search was vulnerable to SQL Injection.
- **A07:2021 - Identification and Authentication Failures:** The initial compromise vector was a stolen JWT (jwt_token_1523_stolen), likely obtained via the observed phishing campaign.

MITRE ATT&CK Framework:

- **T1078 (Valid Accounts):** The entire operation was conducted using the legitimate, stolen credentials of user_1523.
- **T1190 (Exploit Public-Facing Application):** The SQLi attack on the /dashboard/search endpoint.
- **T1087.004 (Account Discovery: Cloud Account):** The IDOR iteration (/api/v1/portfolio/1524...) is a form of account and data discovery.
- **T1595.002 (Defense Evasion: WAF Bypass):** The use of the obfuscated MySQL-specific comment payload /*!50000OR*/ to evade WAF detection rules.
- **T1530 (Data from Cloud Storage):** The exfiltration of the 892KB CSV file from the /dashboard/export endpoint.

1.3 Root Cause Analysis

1. **Critical API Design Flaw:** The API development team relied solely on authentication (a valid token) and built no authorization logic. This allowed any authenticated user to access any other user's data by guessing their ID. This is the most severe flaw.
2. **Insecure Web Development:** The web development team failed to use parameterized queries (prepared statements) for the ticker parameter, allowing for a classic SQLi vulnerability.
3. **WAF Misconfiguration:** The SecOps team misconfigured the WAF. Rule 981001 (Suspicious SQL Pattern) was set to DETECT-ONLY instead of BLOCK. A WAF in "detect-only" mode is merely a logger, not a preventative control.
4. **Token Compromise:** The jwt_token_1523_stolen was compromised, likely via the phishing campaign launched from the same attacker IP.

1.4 Impact Assessment

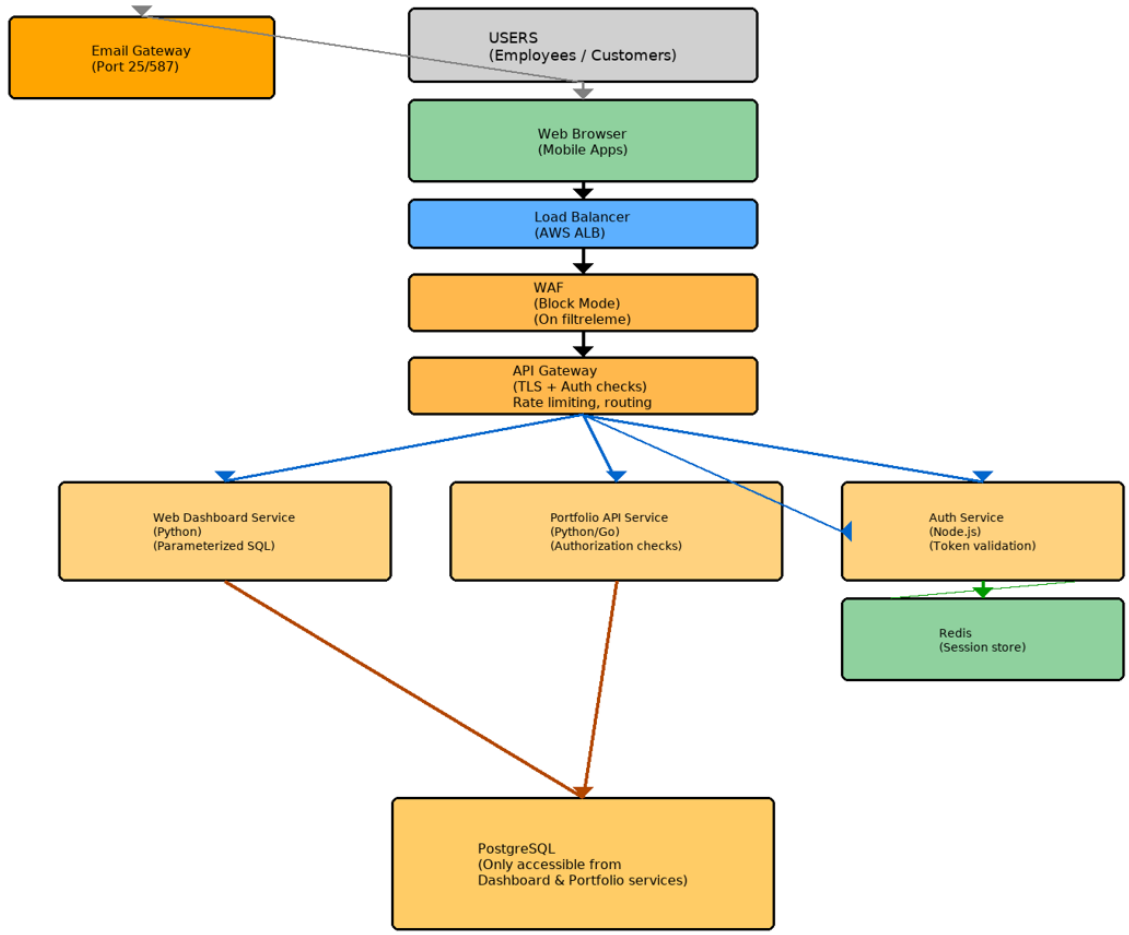
- **Data Breach:** Confirmed exfiltration of full portfolio details for 16 users (via API) and one 892KB CSV file (via Web), which likely contains a significant portion of the user database.
- **Compliance Failure:** This incident likely constitutes a severe breach under **GDPR** (requiring notification within 72 hours) and **PCI-DSS** (if any financial data was present in the database).
- **Reputational Damage:** Loss of customer trust due to compromised financial data.
- **Systemic Risk:** The WAF bypass and IDOR vulnerabilities indicate systemic, critical flaws that likely exist across other endpoints.

Section 2: Architecture Review

2.1 Current Architecture Weaknesses

1. **Flat Trust Model:** The current architecture operates on a "hard shell, soft center" model. Once authenticated (like `user_1523`), the API assumes the user is trustworthy and grants excessive access (e.g., to other users' data).
2. **No API Gateway / Central Authorization:** The Mobile API (`/api/v1/`) and Web Dashboard appear to be separate applications. There is no central API Gateway to enforce global authorization rules, rate-limiting, and logging.
3. **WAF as a Crutch:** The architecture relies heavily on the WAF to protect an insecure application (the SQLi-vulnerable dashboard). When the WAF was bypassed, there was no other layer of defense.
4. **Monolithic Service Logic:** The API's failure to distinguish between *authentication* and *authorization* suggests monolithic code where these concerns are not properly separated, making vulnerabilities hard to spot and fix.

2.2 Improved Security Architecture Diagram



2.3 Recommended Security Controls

1. **API Gateway:** Implement an API Gateway (e.g., Kong, Tyk, AWS API Gateway) to act as the single entry point. It will handle all authentication, global rate-limiting, and basic authorization checks *before* a request ever reaches an application.
2. **Zero Trust Authorization:** The API Gateway checks *authentication*, but each individual microservice (like the "Portfolio API") **must** perform its own *authorization* check (e.g., `if token.UserID != requested.UserID`). Never trust a request, even if it comes from the Gateway.
3. **WAF in Block Mode:** All security rules, especially for Injection, must be set to `BLOCK`, not `DETECT`.
4. **SAST/DAST Tooling:** Integrate Static (SAST) and Dynamic (DAST) analysis tools into the CI/CD pipeline to catch vulnerabilities *before* they reach production.

2.4 Defense-in-Depth Strategy

The proposed architecture moves from a single-layer defense (WAF) to a multi-layer **Defense-in-Depth** model:

- **Layer 1 (Edge):** The **WAF** blocks known bad traffic.
- **Layer 2 (Gateway):** The **API Gateway** validates identity and basic permissions.
- **Layer 3 (Application):** The **Application Code** itself (Web App, Portfolio API) re-validates authorization (Zero Trust) and uses secure coding (parameterized queries).
- **Layer 4 (Database):** The database itself should have hardened permissions.

If the attacker bypasses the WAF (Layer 1), they are stopped by the Gateway (Layer 2). If they have a valid token (bypassing Layer 2), they are stopped by the Application Logic (Layer 3) when they try to access data that isn't theirs.

Section 3: Response & Remediation

3.1 Immediate Actions (0-24 Hours)

1. **Containment:**
    - [DONE] Add `203.0.113.45` to the global firewall blocklist.
    - [DONE] Revoke `jwt_token_1523_stolen` and all active sessions for `user_id=1523`.
2. **Investigation:**
    - [DONE] Force password reset for `user_1523`.
    - [OPEN] Contact `user_1523` to determine the vector of the token theft (confirm phishing).
3. **Triage:**
    - [OPEN] Immediately audit all other API endpoints for the same IDOR vulnerability.

3.2 Short-Term Fixes (1-2 Weeks)

1. **API Team:** Patch the `/api/v1/portfolio/` endpoint to enforce server-side authorization (`if token.UserID != requested.UserID { return 403 }`).
2. **Web Team:** Patch the `/dashboard/search` endpoint. All database queries **must** be rewritten using parameterized queries (prepared statements).
3. **SecOps Team:** Re-configure all WAF rules (especially `981001`) from `DETECT` to `BLOCK` mode.

3.3 Long-Term Improvements (1-3 Months)

1. **Architecture:** Begin planning and migration to the proposed **API Gateway** architecture.
2. **Secure SDLC:** Conduct a full, third-party security code review of all API and web endpoints. Integrate SAST/DAST tools into the CI/CD pipeline.
3. **Training:** Conduct mandatory, targeted training for all employees on phishing and credential hygiene.
4. **Monitoring:** Implement enhanced monitoring for API endpoints to detect anomalous access patterns (e.g., one user accessing many other users' data in a short time).

3.4 Compliance Considerations

- **Breach Notification:** Legal and Compliance teams must be engaged immediately to determine notification obligations under **GDPR** (for EU users) and **CCPA** (for California users). The 72-hour clock for GDPR started when the incident was discovered.
- **PCI-DSS:** If the compromised database contained any payment card information (even partially), this is a critical PCI-DSS failure that requires an immediate report to the acquirer and forensic investigation.